Jev's Structured Outputs Do Not Imply Compatible Uncertainty: An Empirical Study of Confidence Signals in a Multi-Step Decision Workflow

Thejas P Rao
thejasrao262003@gmail.com

Tushar M
tusharm3040@gmail.com

Abstract

Structured-output decision APIs return well-typed answers together with numeric confidence, and it is tempting to treat those numbers as interchangeable probabilities that can be composed across the steps of a workflow. We examine that practice for Jev, TypeSafe's structured decision model, and its three decision primitives: a categorical `choice`, a binary `noul` and an ordinal `score`. TypeSafe documents Jev's confidence as a summary of the output distribution rather than a calibrated estimate of accuracy. We measure what that distinction means once the primitives are composed. The primitives are chained into a three-step triage workflow (route, assess, rate severity). We evaluate on controlled constructed items, on 202 Mozilla bugs labeled by triage and on 240 Linux kernel bugs labeled by their reporters, using 4,584 API calls in the main real-data runs. Three findings hold on both trackers. First, the primitives' confidence signals have distinct failure modes. `choice` confidence is saturated (85–96% of real bugs at ≥ 0.90) yet discriminative (AUROC 0.90 [0.83, 0.96] and 0.98 [0.95, 1.00]). `noul`'s confidence-like value is uninformative (AUROC 0.53 [0.42, 0.65] and 0.41 [0.32, 0.50]). `score`'s confidence is flat across true severity levels. Second, fixed confidence thresholds designed on constructed data lose their intended selectivity on real text: a cut that escalated 73% of constructed items (target 67%) escalated 88% and 78% of real bugs (targets 41–42%). Third, `score` compresses severity toward the middle level, consistent with central-tendency bias reported for LLM raters in other domains, and its confidence gives no indication of it. In exploratory analyses, the composition rules we tested did not consistently identify which cases to escalate: ranking escalations by composed confidence did no better than random selection on either tracker (one borderline worse-than-random estimate did not replicate), and the intervals cannot exclude a modest signal. A natural comparison of confidence-based against decision-based composition turns out to be confounded when the target action is itself a function of the decisions. Pre-registered tests then showed three further things. First, on 500 new bugs, a frozen estimator built on what the model decided ranked wrong workflow actions better than final-step confidence (AUROC 0.76 vs. 0.55). Second, this held across five evaluated models from four sources, including an independently trained Jev-style model and three general instruct models. The signal was mostly the chosen action's historical error rate, and failure modes differed by model family. Third, at a 10% review budget this signal did not reduce cost relative to confidence-ranked review. An oracle shows why: the expensive errors exist but cannot be isolated within an action.

1 Introduction

A model can return a perfectly valid, well-typed structured response and still make the wrong decision. When several such decisions are chained into a workflow, errors at one step propagate to the next. A common hope is that each step's reported confidence can be composed into a workflow-level signal that says when to hand a case to a human.

We ask a narrower and more testable question. *When a workflow chains heterogeneous decision primitives, can it rely on their uncertainty signals, and how should it combine them?* Heterogeneity matters because Jev exposes three primitives whose uncertainty is *structurally* different. `choice` returns a distribution over labels plus a scalar confidence. `score` returns an ordinal distribution plus a scalar confidence. `noul` returns a single probability with no separate confidence field. Treating these numbers as one kind of quantity is a modeling assumption, and this paper tests it.

What TypeSafe documents. TypeSafe's documentation describes Jev's `confidence` for `choice` and `score` as a number that "collapses" the output distribution into a value between 0 and 1. It is 1 when all probability is on one outcome and 0 when probability is spread evenly. The documentation calls this "one reasonable way to summarize a distribution, not the only one", and advises setting thresholds from one's own data [28]. For `noul`, it returns "a single probability" with "no separate `confidence`". Our results are therefore not a test of a calibration claim. They are a measurement of how these documented concentration statistics behave as escalation signals when Jev's primitives are chained, which the documentation leaves open.

Contributions.

1. **Characterization** (Section 4). Per-primitive behavior of confidence on controlled and real data, showing three incompatible profiles.
2. **Transfer** (Section 5). Fixed thresholds set on constructed content lose their selectivity on real text, in two distinct ways.
3. **Real-data validation** (Section 6). Two bug trackers from different projects, with different labeling processes (triage and reporter), on which severity compression appears in both.
4. **Composition** (Section 7, exploratory). The confidence-based escalation rules we tested did not consistently pick better cases than random on either tracker. We identify, and correct for the follow-up, a confound in the obvious comparison.
5. **Held-out and cross-model tests** (Sections 8 and 9, pre-registered). A frozen decision-feature estimator ranks wrong workflow actions better than final-step confidence on 500 new bugs. The gap replicates across five evaluated models from four sources. Most of the signal is the chosen action's historical error rate.
6. **Intervention** (Section 10, pre-registered). At a 10% review budget, that predictive signal does not yield a lower cost than confidence-ranked review. An oracle shows the reason: the expensive errors exist but cannot be isolated within an action.
7. **Method** (Section 11). A pre-registration and audit process that caught six design or implementation defects, most of them before any evaluation data existed.

Non-claims. We make no claim about general calibration, other Jev versions or other APIs, domains beyond software bug triage, or real-world operational benefit.

2 Related work

Calibration of single decisions. Post-hoc calibration [22, 24] and the miscalibration of modern networks [16] are well studied, and confidence estimation for language models has been surveyed in [14].

Selective prediction and deferral. Risk–coverage trade-offs and reject options [11–13], learning to defer [21] and prediction sets in human–AI teams [5] address *one* decision.

Conformal methods and composition. Conformal prediction [3] and conformal risk control [4] give distribution-free guarantees. Composition across stages has been studied for two stages [34] and for multi-stage NLP and LLM pipelines, where errors compound across stages and independently calibrated stages lack a joint coverage guarantee [19]. Error propagation through pipeline steps has also been quantified [8]. Classical reliability theory composes component reliabilities [6].

Cascades and escalation. Cascades route a *single* query through model tiers. Prior work studies early abstention [31], the optimal depth and deferral thresholds of cascades [7], conformal cascades [9], confidence tuning [26] and judges with escalation guarantees [17].

Combining confidence across components. Fixed combination rules such as the product rule are classical [18]. The product rule's usual justification assumes conditional independence and comparable probability scales, and these are precisely the assumptions our strategy S2 makes implicitly. At the system level, a recent taxonomy argues that uncertainty in LLM applications must be studied across the whole compound system, as it propagates within models, across system components and through socio-technical processes [30]. Cost-sensitive deferral to multiple experts under workload limits [2] is closely related to the budgeted escalation setting our follow-up uses.

Ordinal outputs and central tendency. LLM raters have been reported to compress ordinal scores toward the middle of the scale, for example in clinical scoring, where prompt changes did not remove the effect [32]. Few-label calibration of ordinal LLM outputs models such flattening explicitly [29]. Our severity result is consistent with this literature. What we add is that it appears in bug triage, against two labeling processes, and that the primitive's own confidence gives no sign of it.

Agents and trajectories. Closest to us, a counterexample shows that step-level calibration need not imply trajectory-level calibration for LLM agents, and step errors are empirically coupled along trajectories [20]. Related work calibrates agent trajectories [33] and studies uncertainty propagation in multi-step agentic decisions [10].

Positioning. Prior work studies calibration of single decisions, cascades of the *same* decision, agent trajectories, and system-level propagation in general. Our contribution is narrower and empirical. We measure how the uncertainty signals of three *structurally different* primitives of Jev behave inside one workflow against real labels, and whether simple composition rules over them identify escalation candidates. We are not aware of prior work with this specific design. Our literature search was web-based and did not systematically cover reliability engineering or HCI escalation research, so we make no stronger priority claim.

3 Setting

Jev and repeats. All calls go to the TypeSafe API with model `jev-latest`, which resolved to `jev-1.13.0` in every one of our calls. Each node is called three times, and the three answers are reduced to one decision. For `choice` and `score` we take the modal decision with a tie tolerance

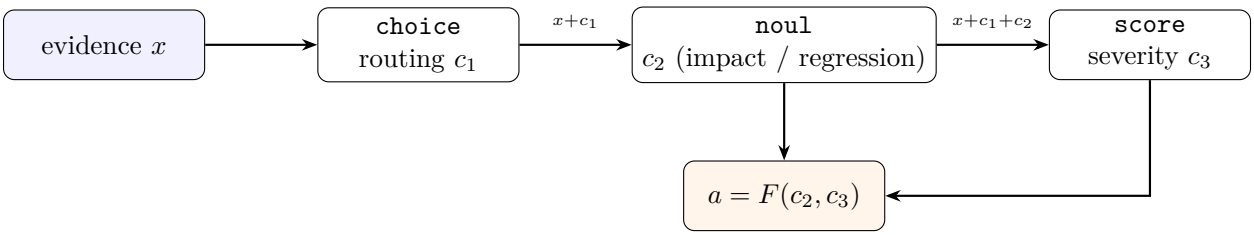


Figure 1: The three-step workflow. Each node is called three times. The action rule F uses only c_2 and c_3.

of 0.02; a node with no two-of-three majority is *repeat-unstable*. For `noul` we threshold the mean probability at 0.5, and its confidence-like value is $2\,|\bar{p} - 0.5|$.

Workflow. Node 1 (`choice`) routes the incident to a network or storage subsystem. Node 2 (`noul`) assesses a binary property: customer impact on constructed data, regression on real bugs. Node 3 (`score`) rates severity on three levels. Each node reads the evidence plus the upstream decisions, appended by a fixed adapter (Fig. 1).

Target action. The target is $a^* = F(c_2^*, c_3^*)$, where F maps an *unclear* c_2 to *request more evidence*, $c_3 = 2$ to *escalate*, $c_3 = 1$ with $c_2 = 1$ to *defer*, and everything else to *accept* (Table 4). F is a construct we defined, not an independently validated outcome, and every strategy result is conditional on it.

Strategies.

- **S1:** F applied to the node decisions.
- **S4a:** S1 plus escalation on repeat instability.
- **S2:** escalate the $\lceil qn \rceil$ instances with the lowest product of per-node confidences and accept the rest. Here q is the share of non-accept targets.
- **S5:** the same, ranked by per-primitive rank-normalized uncertainty.
- **2h / 5h:** S1 plus confidence-ranked escalation of a fixed 10% budget.
- **R:** the same budget escalated at random.

Statistics. Agreement rates carry Wilson 95% intervals. AUROCs use the Mann–Whitney estimator, with 95% percentile bootstrap intervals over bugs ($B = 2{,}000$, fixed seed). Bugs are distinct reports with no shared unit, so we resample bugs. Paired strategy comparisons use the exact McNemar test on discordant pairs, and the difference in $P(Y_{\text{decision}})$ gets a bootstrap interval. Within every resample, the set-relative rankings (S2, 2h) are recomputed. We compare the two trackers through the difference of their paired differences, with independent bootstraps per tracker. We do not compare their p-values. For budgeted overrides, the reference is the *exact* expected cost of k random overrides. The randomization p-value (10,000 random k-subsets) is conditional on the evaluated bugs, and a bootstrap interval over bugs gives the sampling uncertainty. An interval that covers the null value is read as *no evidence of an effect*, not as evidence of no effect. The scripts are `paper_statistics.py` and `mozilla_exploratory_analysis.py`.

Table 1: Per-primitive behavior of the confidence signal. AUROC is for predicting the node's own agreement with the reference label, with 95% bootstrap intervals.

	Constructed items	Mozilla (n=202, triage)	Kernel (n=240, reporter)
`choice`	High-confidence error subgroup: mean confidence 0.915, accuracy 0.333 (n=6)	85% at ≥ 0.90, agreeing 0.96; AUROC 0.90 [0.83, 0.96]; ECE 0.05	96% at ≥ 0.90, agreeing 0.99; AUROC 0.98 [0.95, 1.00]; ECE 0.02
`noul`	24/24 correct, confidence below accuracy in every bin	Agreement flat across bands (0.71–0.84); AUROC 0.53 [0.42, 0.65]; ECE 0.36	Non-monotonic (0.58–0.93); AUROC 0.41 [0.32, 0.50]; ECE 0.36
`score`	Tracks the most severe sentence (6/6); confidence varies by 0.173 with sentence order alone (1 of 3 items)	Confidence ≈0.6 at every true level; AUROC 0.58 [0.50, 0.66]	Confidence 0.65 / 0.62 / 0.61 by true level; AUROC 0.57 [0.50, 0.64]

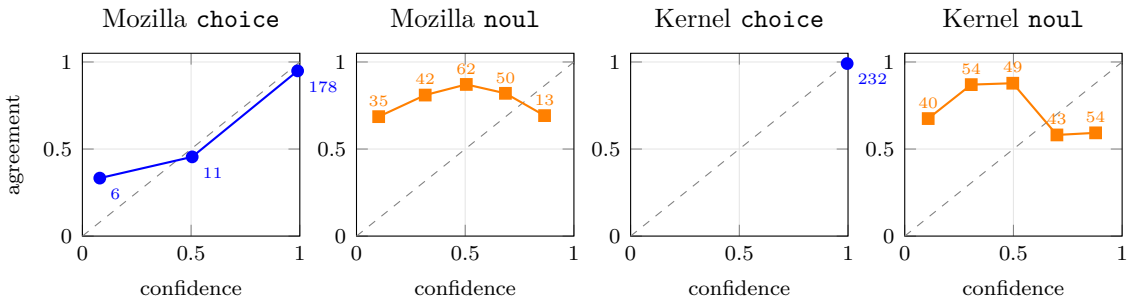


Figure 2: Reliability diagrams (five equal-width confidence bins, bins with $n \geq 5$; the label is n; dashed line: perfect calibration). `choice` puts nearly all bugs in its top bin, and that bin is accurate. `noul`'s confidence-like value $2|\bar{p} - 0.5|$ sits far *below* its agreement (underconfidence), and agreement does not rise with it.

Outcomes. Outcomes are $P(Y_{\text{decision}})$, the probability that the action equals a^*, and the cost under a frozen matrix (Table 5), in which a wrong *accept* costs 6–10, any other mismatch 1, and each escalation an extra 1 for human time.

4 Characterization: incompatible uncertainty semantics

Table 1 and Fig. 2 show three incompatible profiles. `choice`'s confidence is almost always near 1.0. It still discriminates, because the rare low-confidence answers are where its errors are. The high-confidence error subgroup found on deliberately ambiguous constructed items does not recur on real text (2 of 6 kernel errors are at ≥ 0.90). `noul`'s confidence-like value carries no usable information about correctness on either tracker. Its Mozilla interval covers 0.5, and its kernel interval lies at or just below 0.5 (weakly anti-informative). `score`'s confidence does not vary with the true level, so it gives no warning of the systematic bias in Section 6. Any composition rule that multiplies or averages these numbers inherits this mix.

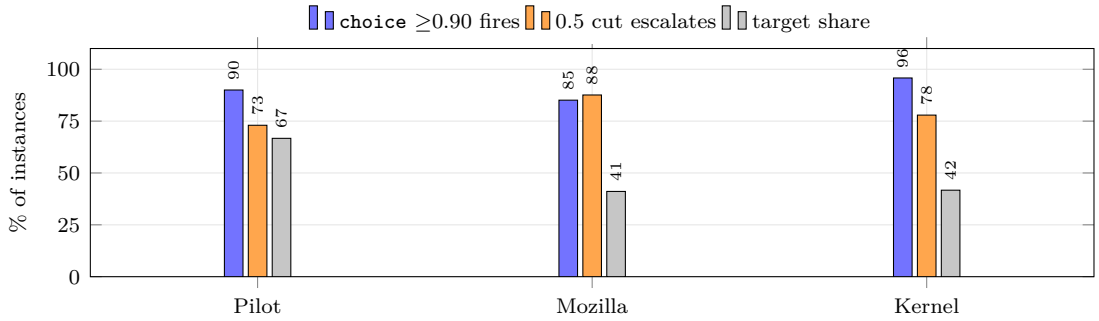


Figure 3: Selectivity of the two fixed-threshold rules, set on constructed data and measured on each rule's own input signals. The trigger fires on 85–96% of instances everywhere. The product cut is near its target on the constructed pilot and far above it on real text.

Table 2: Node agreement with reference labels (Wilson intervals). Each baseline is the most frequent label.

	Mozilla		Kernel	
Node	Agreement [95% CI]	Baseline	Agreement [95% CI]	Baseline
`choice`	0.89 [0.83, 0.92]	0.59	0.975 [0.95, 0.99]	0.50
`noul`	0.80 [0.74, 0.85]	0.74	0.725 [0.67, 0.78]	0.81
`score`	0.53 [0.46, 0.60]	0.40	0.44 [0.38, 0.50]	0.33

5 Fixed thresholds lose their selectivity on real text

The rules in Fig. 3 fail in two different ways. On real text the trigger fired on 85% (Mozilla) and 96% (kernel) of bugs, and the product cut escalated 88% and 78% against targets of 41% and 42% (constructed pilot: 27/30 fired; 73% escalated against 67%). The `choice` trigger is non-selective *everywhere*, because `choice` confidence is saturated: that is a design failure, visible only once measured. The product cut is *distribution-sensitive*. It was near its target on constructed data and far from it on real data, because real text pulls `noul` toward 0.5 and so pulls the product down. A fixed threshold carries the confidence scale of the data it was set on, and our constructed data was too clean to reveal this. The two Revision-10 strategies built on these thresholds scored 0.37–0.40 (Mozilla) and 0.34–0.37 (kernel). We replaced them with rank-based rules at a matched escalation rate before any evaluation.

6 Real-data validation on two trackers

Data. *Mozilla:* 202 fixed Firefox networking and storage bugs. Severity S1–S4 is assigned by triage and mapped to levels 2/2/1/0, and the regression flag is a keyword. Bugs whose severity was revised were excluded. *Linux kernel:* 240 fixed networking and storage bugs. Severity (blocking/high/normal/low → 2/2/1/0) is set by the reporter, and the regression flag is a dedicated field. Severity is rarely changed after filing: 15 of the 240 sampled bugs, all moving away from the default "normal". Both samples are balanced at 40 per (subsystem, level) cell where available. Texts are cleaned, privacy-scrubbed and truncated to 2,000 characters. Agreement is with the reference label, not ground truth.

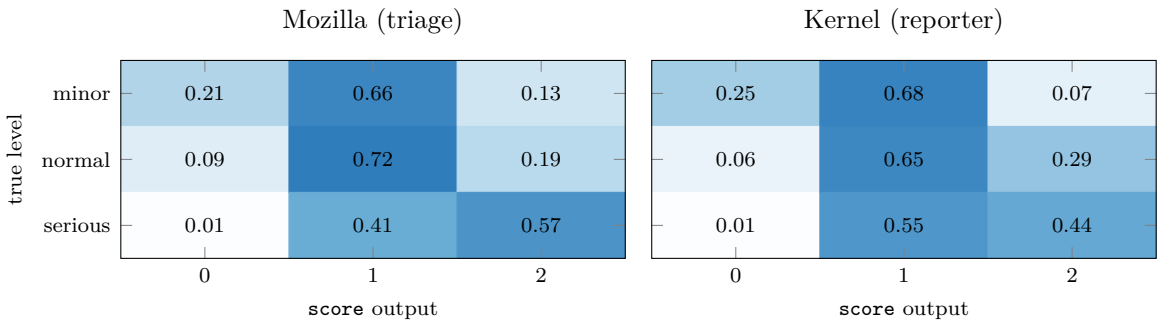


Figure 4: Row-normalized confusion of `score` against reference severity (repeat-unstable outputs excluded). On both trackers the middle column absorbs mass from both extremes: minor bugs are rated up, serious bugs down.

Routing and regression. `choice` agrees with the component label well above baseline on both trackers (Table 2). It still agrees 0.84 on Mozilla texts with no subsystem keyword. `noul`'s errors change direction between trackers. On Mozilla it mostly misses regressions (23 misses vs. 17 false alarms). On the kernel it over-flags them (63 false alarms vs. 3 misses), which leaves it below the always-"no" baseline. The kernel false-alarm rate is the same on bugs explicitly marked "not a regression" (33/100) as on bugs with the field unset (30/94). So unset labels do not explain it, and on bugs with a set field `noul` beats its baseline (110/146 vs. 100/146).

Severity compression. Figure 4 shows the row-normalized confusion of `score`. On both trackers, most minor bugs are rated higher than their label: 79% [0.67, 0.88] on Mozilla and 75% [0.65, 0.83] on the kernel. A large share of serious bugs are rated lower: 43% and 56%. The middle level absorbs both. With Mozilla's own severity definitions in place of ours, serious bugs improve but normal bugs are pushed up, and agreement moves only from 0.53 to 0.56. The criteria matter, but the compression persists. The two trackers differ in project, bug population and labeling process. They share Jev, the prompts, our sampling design and, plausibly, training-data exposure, so the replication speaks against an artifact of one *labeling* process, not against every shared factor. The pattern matches central-tendency bias reported for LLM raters in other domains [32].

7 Composition (exploratory)

All analyses in this section were specified after the Mozilla results had been seen. They are hypothesis-generating. The pre-registered tests that followed are reported in Sections 8 to 10.

The naive comparison is confounded. On Mozilla, decision composition (S1) beats confidence composition (S2) by 0.17 [0.08, 0.27] in $P(Y_{\text{decision}})$, with 61 vs. 27 discordant pairs (exact McNemar $p = 0.0004$). But S2 never reads the node decisions, while a^* escalates on the severity *decision*. To be right on a serious bug, S2 needs low confidence to *mean* high severity, so the comparison mostly measures whether confidence can stand in for severity. On the kernel the difference is smaller and inconclusive: 0.06 [−0.03, 0.15], 66 vs. 51 discordant pairs, $p = 0.20$. The difference between trackers is 0.11 [−0.01, 0.24]. Its interval covers zero, so we have *no evidence that the effect differs* between trackers, only a weaker estimate on one of them.

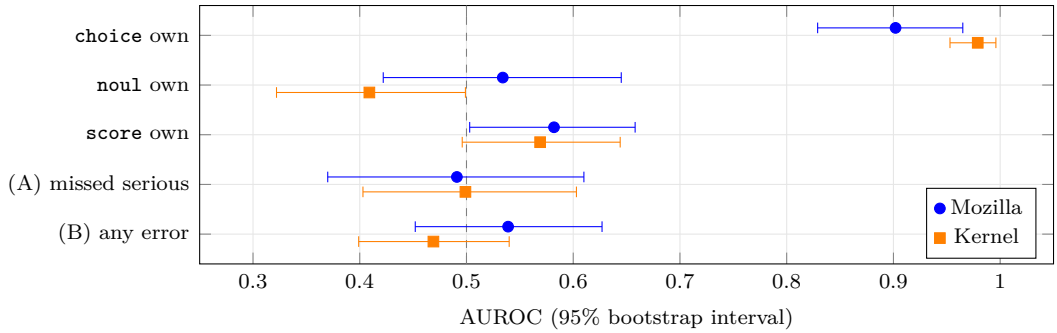


Figure 5: Every AUROC in the paper with its interval; dashed line: chance. Only `choice`'s confidence discriminates its own errors. Composed confidence is at chance for both escalation targets, (A) and (B).

The fair question, and two different targets. Does confidence add anything *beyond* the decisions? Starting from the decision-based action F, we also escalate the k least-confident bugs that F did not escalate, for every k, and compare with the exact expectation under random selection (Fig. 6). We report two AUROCs with different prediction targets, which should not be conflated:

- **(A) Missed serious bugs.** Among bugs that F did not escalate, does low composed confidence rank those whose target is *escalate* first? This is what an override budget can fix. AUROC 0.49 [0.37, 0.61] on Mozilla and 0.50 [0.40, 0.60] on the kernel.
- **(B) Any decision error.** Across all bugs, does low composed confidence rank those where F's action is wrong first? AUROC 0.54 [0.45, 0.63] on Mozilla and 0.47 [0.40, 0.54] on the kernel.

Every interval covers 0.5 (Fig. 5). The data show no evidence that composed confidence ranks either target better than chance, but they cannot rule out a modest signal (AUROC up to about 0.6).

Budgeted overrides, scored by cost. At a fixed budget of $k = \lceil 0.1n \rceil$ overrides ($k = 21$ on Mozilla, 24 on the kernel), let T be the total cost of the confidence-ranked overrides and μ the exact expected total under k overrides drawn uniformly from the same candidates. Two analyses answer two different questions.

- **Randomization test, conditional on the evaluated bugs.** This asks whether the ranking selected worse or better overrides than random selection *on these bugs*. The statistic is $|T - \mu|$. The null distribution comes from 10,000 random k-subsets of the candidates, and the test is two-sided.
- **Bootstrap over bugs.** This asks how stable the per-bug difference $(T - \mu)/n$ is under resampling of the bugs, with rankings and μ recomputed in each resample.

On Mozilla, $T = 390$ against $\mu = 357.2$. The observed value sits at the edge of the null distribution (2.5–97.5% range 323–388), so $p = 0.05$. The per-bug difference is +0.16 [−0.01, 0.24], and 97% of resamples are positive. That is borderline evidence that the product ranking chose *worse* overrides than random on Mozilla. On the kernel, $T = 338$ against $\mu = 358.6$ ($p = 0.18$), a per-bug difference of −0.09 [−0.23, 0.06]: inconclusive, and in the opposite direction. These analyses are exploratory

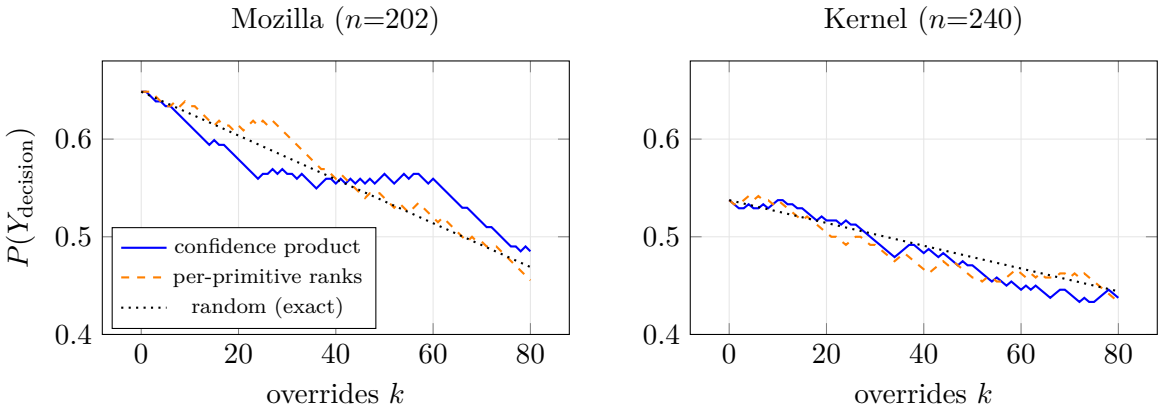


Figure 6: Adding confidence-ranked escalations on top of decision composition. At $k = 0$ the curve is S1. Neither ranking consistently improves on random selection of the same number of cases: deviations are a few points at most and change sign between rankings and trackers. For example, per-primitive ranks run slightly above random on Mozilla for $k \approx 15$–35, and the confidence product runs below.

and uncorrected for multiplicity. We read them as no consistent benefit from confidence-ranked overrides, with one borderline harmful estimate that did not replicate, and not as an established harm.

A structural reason. In this workflow the only discriminative confidence (`choice`, AUROC 0.90–0.98) belongs to the one node that F does not use. The informative signal never reaches the action, and the two signals that do reach it are close to uninformative.

8 A pre-registered held-out test

Design. The exploratory analyses above suggested that a model's *decisions* carry the information its confidence lacks. We tested this prospectively, as follows.

- **Estimator.** We trained one decision-feature estimator (E_{dec}: L_2-regularized logistic regression on the `score` level, the `noul` decision and the workflow action, with $\lambda = 1$) on all 442 previously collected bugs. We froze it by hash, before any new data existed.
- **Fresh bugs.** We collected 500 new bugs: 250 per tracker, sampled at each tracker's *natural* mix, with no overlap with the training bugs. That required 4,500 Jev calls.
- **Hypotheses (pre-registered).** HR1: E_{dec} ranks wrong workflow actions better than low final-step confidence. HR2: E_{dec} ranks above chance on each tracker. One-sided bootstrap tests, Holm-corrected.

Result. Both hypotheses are supported.

- **HR1:** E_{dec} reaches AUROC 0.763 against 0.549 for final-step confidence, a difference of +0.214 (one-sided 95% lower bound +0.158).
- **HR2:** AUROC is 0.811 on Mozilla (lower bound 0.761) and 0.697 on the kernel (lower bound 0.643).

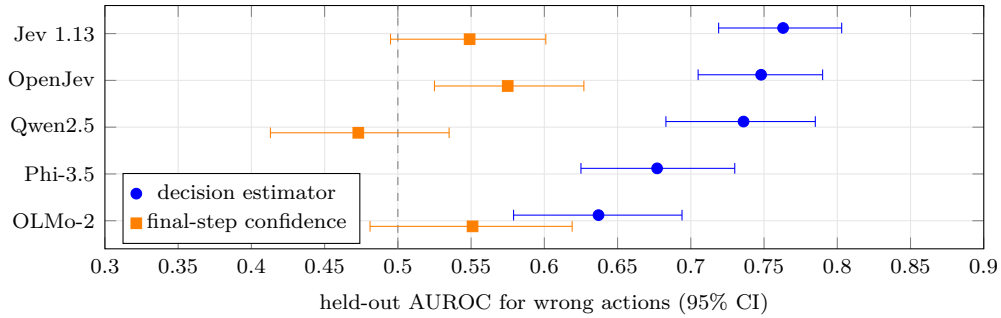


Figure 7: Across five models, the decision-feature estimator ranks wrong workflow actions better than the model's own final-step confidence. The intervals overlap only for OLMo-2.

- **Calibration:** the estimator's predicted failure rate matches the observed rate overall. But it outputs only 7–8 distinct risk levels (ECE 0.18–0.20).

9 Cross-model replication

Models. To test whether the result is specific to Jev, we ran the same 942 bugs, questions and three-step adapter through four further models.

- **OpenJev 27B** [23]: an independently trained Jev-style decision model. It is served on one H100 exactly as its authors specify, with their calibrated readout. Its 2,826 requests had 0 failures. (An unofficial architectural reconstruction of Jev also exists [15], but it has no trained weights, so it could not be evaluated.)
- **Three general instruct models:** Qwen2.5-7B [25], OLMo-2-7B [27] and Phi-3.5-mini [1]. Each runs behind a Jev-compatible readout: next-token probabilities over the allowed labels, with TypeSafe's documented confidence formulas.

Each model's estimator was trained on that model's own outputs for the 442 training bugs and tested on its 500 held-out bugs. The pre-registered tests mirror HR1 and HR2, with Holm correction across all 8 tests.

Deviations, disclosed. We recorded three pre-registration amendments.

- OpenJev was added before any of its outputs existed.
- OLMo-2 needed a newer `transformers` release.
- A readout bug, in which a SentencePiece word-start token was shared by all digit labels, was found after interim results had been seen. All three general models were re-run under one corrected readout. The conclusions did not change.

Result. All 8 pre-registered tests are supported (Fig. 7).

- **Decision estimators:** AUROC 0.64–0.76.
- **Final-step confidence:** 0.40–0.58.

- **Weakest model:** OLMo-2 (0.637 vs. 0.551; one-sided $p = 0.037$, passing at the last Holm step).
- **Transfer:** Jev's frozen estimator, never refitted, reaches 0.76 [0.72, 0.80] on OpenJev's outputs, as well as OpenJev's own estimator does.

The signal is simple. A class-frequency rule, "the historical error rate of the action the workflow chose" (estimated on training bugs only), matches the estimator for Jev (0.76) and OpenJev (0.75), and comes close for the others (0.58–0.70). Single uncertainty signals each sit at chance for workflow failure:

- routing confidence: 0.48–0.52;
- regression confidence: 0.45–0.56;
- severity entropy: 0.40–0.55.

The predictive information is therefore mostly *which action was chosen* and how often that action has been wrong before.

Failure modes differ by model family.

- **Jev-style models (Jev and OpenJev):** both pull severity to the middle level.
- **General instruct models:** they push severity to "blocking" for 67–100% of bugs, whatever the true level, and are wrong far more often (72–80% vs. 43–47%).

The absolute failure rates are not a clean quality comparison, because the two groups use different interfaces. The within-model contrast is.

10 Does the predictive signal reduce cost?

Design. We pre-registered the test before computing any cost outcome.

- **Budget:** 50 reviews (10%) per model on the held-out bugs. A review escalates the case to a human, priced with the frozen cost matrix.
- **Primary arm (A1):** rank candidates by the *action-level expected saving* of a review, estimated on training bugs. It is cost-aware rather than error-rate-based, because a wrong *defer* costs no more than an escalation.
- **Comparators:** confidence-ranked review (B) and random review (C).

Result. Two of ten pre-registered tests are supported: A1 beats random for Jev and OpenJev, by about 10–12 cost units. A1 never beats confidence. Every review arm costs more than no review at all; for Jev, 658 with no review against 681–714 with review (Fig. 8).

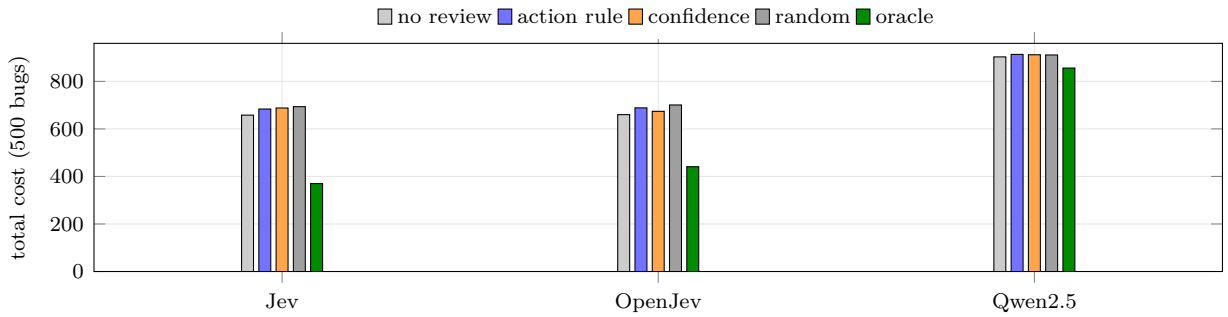


Figure 8: Total cost at a 10% review budget. No feasible strategy beats no review. A perfect reviewer (oracle, an upper bound using held-out outcomes) would cut Jev's cost by 44% and OpenJev's by 33%.

Where the cost is. A descriptive follow-up separates two regimes.

- **Jev-style models:** 40–52% of no-review cost comes from wrongly *accepted* serious or regression bugs. An oracle that picked the right 50 cases would save 33–44%. But those cases are 15–16% of accepted cases. The action rule cannot separate them, because every accept looks alike to it, and confidence cannot either. The value exists; the identification fails.
- **General models:** cost is dominated by over-escalation (*escalate* when the target is *accept*). More review cannot reduce that; even the oracle saves at most 5%.

Ranking which actions are wrong is therefore not the same as finding which reviews save money: frequent errors and expensive errors differ.

11 Method: audits that caught defects

Table 3 lists the defects the study's own audits caught. Most were found before the data that would have exposed them existed. The exception is the confound, found after a result. It would have been caught earlier by asking, for every strategy and every target class: *can this strategy be right here, and through what information?* We recommend this check, together with measuring each trigger's selectivity on pilot inputs, before freezing any escalation policy.

12 Limitations

- **Generality.** Five models in one domain (bug triage) and one workflow. Jev was version 1.13.0. The general instruct models use our prompt and readout, not a trained decision interface. Later versions and other domains may behave differently.
- **References are not truth.** Mozilla's triage sample excludes re-triaged bugs, which biases agreement upward. Kernel labels are reporters' claims, and the kernel's middle level is a default value.
- **Small constructed samples.** The `choice` error subgroup is $n = 6$, and `noul`'s perfect accuracy on constructed data is 24 items.
- **Constructed target.** $a^* = F(\cdot)$ is constructed, and all strategy results are conditional on it.

Table 3: Defects found by the study's own audits.

Defect	Detected by	When
A "primitive-aware" arm structurally identical to the baseline	inspecting the action rule	before evaluation
A trigger firing on ≈90% of inputs	selectivity on input signals only	before evaluation
A threshold sensitive to the data distribution	smoke test vs. pilot	before the full run
Four silent runner defects (e.g. the literal "None" shown to the model; partial data analyzed)	adversarial probing	before the live run
The confidence-vs-decisions confound	post-hoc review	after the Mozilla result
Template content whose labels were recoverable from punctuation alone (100%) and whose sentences leaked cross-label cues	independent review (separate AI agent)	before the controlled study

- **Memorization.** Old public bugs may appear in training data. The recent subsets are small (Mozilla 54, kernel 35).
- **Cross-model deviations.** Three amendments to the cross-model pre-registration, including a readout bug fixed after interim results were seen. On 3% of OLMo-2's requests, most of the probability fell outside the allowed labels; excluding them does not change its result.
- **Intervention scope.** One budget and one cost table, both frozen in advance. Reviewers are assumed correct, and the oracle is an upper bound, not a strategy.
- **Protocol revisions.** The protocol went through twelve revisions. All are dated and disclosed, and each was made before the data it affects existed. The number is nonetheless large.

13 Pre-registered follow-up (planned; not yet run)

No result in this paper comes from the studies in this section.

Case-level expensive-error estimator. The intervention analysis points to a specific next question. Can a per-case estimator of *whether an accepted case is wrongly accepted* capture part of the oracle's 33–44% headroom? It would be developed on the 442 training bugs only, pre-registered, and tested on a fresh batch of bugs, not on the repeatedly analyzed held-out set. Its primary outcome is cost at a fixed budget against confidence, action-rule and random review.

Template study. To test the composition question under control, we pre-registered a study on deterministic template content: 3,103 units, each instance describing one incident about one service, with value-neutral sentence frames and 13 automated audits. Its primary tests are H1, confidence-ranked vs. random overrides at a 10% budget scored by cost, and H2, per-primitive vs. raw-product

ranking, with a sign-flip test clustered by service. Both are Holm-corrected. The sample size is set from a second calibration pilot, with a content ceiling of 720 instances. Given the results above, a null H1 is a live outcome, and it would be reported as the answer to the composition question. Deterministic templates raise their own external-validity questions. That is why the real-data studies above are reported separately, and why their conclusions do not depend on the follow-up.

14 Conclusion

Structured outputs make uncertainty *look* uniform: every Jev primitive returns a number between 0 and 1. In the evaluated workflow, Jev's three primitives showed distinct confidence failure modes, fixed thresholds on them did not transfer, and severity compression appeared on two trackers while the primitive's own confidence gave no sign of it.

Pre-registered tests then showed that what the workflow *decided* predicts its failures better than how confident it was. This held on 500 new bugs and across five evaluated models from four sources, and it was driven mostly by the chosen action's historical error rate. Jev-style decision models and general instruct models failed in different ways: compression versus over-escalation.

That prediction did not become a cheaper review policy. The expensive errors are real, but they hide inside actions that look alike. Knowing which decisions are likely wrong is different from knowing which are worth correcting. Workflow uncertainty has to be evaluated against the decision it is meant to improve.

References

[1] Marah Abdin, Jyoti Aneja, Hany Awadalla, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.

[2] Jean V. Alves, Diogo Leitão, Sérgio Jesus, Marco O. P. Sampaio, Javier Liébana, Pedro Saleiro, Mário A. T. Figueiredo, and Pedro Bizarro. Cost-sensitive learning to defer to multiple experts with workload constraints. *Transactions on Machine Learning Research*, 2024.

[3] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[4] Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. Conformal risk control. In *International Conference on Learning Representations (ICLR)*, 2024.

[5] Varun Babbar, Umang Bhatt, and Adrian Weller. On the utility of prediction sets in human-AI teams. In *Proceedings of IJCAI*, 2022.

[6] Richard E. Barlow and Frank Proschan. *Mathematical Theory of Reliability*. Wiley, 1965.

[7] Dylan Bouchard. Is escalation worth it? On the depth of LLM cascades. *arXiv preprint arXiv:2605.06350*, 2026. v2; v1 was titled "Is Escalation Worth It? A Decision-Theoretic Characterization of LLM Cascades".

[8] Aritra Chowdhury, Malik Magdon-Ismail, and Bülent Yener. Quantifying contribution and propagation of error from computational steps, algorithms and hyperparameter choices in image classification pipelines. *arXiv preprint arXiv:1903.00405*, 2019.

[9] Yifan Dou, Shikan Lian, and Shibo Li. Conformal cascade: Distribution-free accuracy guarantees for multi-tier LLM inference. *arXiv preprint arXiv:2607.25018*, 2026.

[10] Jinhao Duan, James Diffenderfer, Sandeep Madireddy, Tianlong Chen, Bhavya Kailkhura, and Kaidi Xu. UProp: Investigating the uncertainty propagation of LLMs in multi-step agentic decision-making. *arXiv preprint arXiv:2506.17419*, 2025.

[11] Ran El-Yaniv and Yair Wiener. On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11, 2010.

[12] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[13] Yonatan Geifman and Ran El-Yaniv. SelectiveNet: A deep neural network with an integrated reject option. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.

[14] Jiahui Geng, Fengyu Cai, Yuxia Wang, Heinz Koeppl, Preslav Nakov, and Iryna Gurevych. A survey of confidence estimation and calibration in large language models. In *Proceedings of NAACL (Volume 1: Long Papers)*, pages 6577–6595, 2024.

[15] Kye Gomez. open-jev: an unofficial reconstruction of Jev's architecture. `https://github.com/kyegomez/open-jev`, 2026. Untrained architecture with random weights. Accessed 2026-10-17.

[16] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

[17] Jaehun Jung, Faeze Brahman, and Yejin Choi. Trust or escalate: LLM judges with provable guarantees for human agreement. In *International Conference on Learning Representations (ICLR)*, 2025.

[18] Josef Kittler, Mohamad Hatef, Robert P. W. Duin, and Jiri Matas. On combining classifiers. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 20(3):226–239, 1998.

[19] Varun Kotte. PASC: Pipeline-aware conformal prediction with joint coverage guarantees for multi-stage NLP and LLM pipelines. *arXiv preprint arXiv:2605.18812*, 2026.

[20] Moule Lin, Qizhen Lan, Shuhao Guan, Weipeng Jing, Jiexin Fan, David Gregg, and Goetz Botterweck. Uncertainty quantification for LLM agents: A taxonomy, an evaluation protocol, and an empirical study. *arXiv preprint arXiv:2609.07395*, 2026.

[21] David Madras, Toniann Pitassi, and Richard Zemel. Predict responsibly: Improving fairness and accuracy by learning to defer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[22] Mahdi Pakdaman Naeini, Gregory Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2015.

[23] OpenJev. OpenJev: an open-weights decision model. `https://huggingface.co/openjev/openjev`, 2026. CC BY-NC 4.0 weights; independent of TypeSafe. Accessed 2026-10-17.

[24] John C. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. Technical report, Microsoft Research, 1999.

[25] Qwen, An Yang, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[26] Stephan Rabanser, Nathalie Rauschmayr, Achin Kulshrestha, Petra Poklukar, Wittawat Jitkrittum, Sean Augenstein, Congchao Wang, and Federico Tombari. Gatekeeper: Improving model cascades through confidence tuning. *arXiv preprint arXiv:2502.19335*, 2025.

[27] Team OLMo, Pete Walsh, Luca Soldaini, et al. 2 OLMo 2 furious. *arXiv preprint arXiv:2501.00656*, 2024.

[28] TypeSafe. Confidence. TypeSafe documentation, `https://docs.typesafe.ai/confidence`, 2026. Accessed 2026-10-16.

[29] Xiangwei Wang, Peng Wang, and Saman Halgamuge. CORDIAL: Calibrating ordinal LLM outputs from few labels. *arXiv preprint arXiv:2609.29807*, 2026.

[30] Boming Xia, Liming Zhu, Erdun Gao, Qinghua Lu, Minhui Xue, and Dino Sejdinovic. Uncertainty propagation in LLM-based systems. *arXiv preprint arXiv:2604.23505*, 2026.

[31] Michael J. Zellinger, Rex Liu, and Matt Thomson. Cost-saving LLM cascades with early abstention. *arXiv preprint arXiv:2502.09054*, 2025.

[32] Jiaqing Zhang, Sandeep Elluri, Bhanu Cherukuvada, Yonah Joffe, Jessica Sena, Miguel Contreras, Scott Siegel, Subhash Nerella, Catherine Price, and Parisa Rashidi. Auditing multimodal LLM raters: Central tendency bias in clinical ordinal scoring. *arXiv preprint arXiv:2605.16386*, 2026.

[33] Jiaxin Zhang, Caiming Xiong, and Chien-Sheng Wu. Agentic confidence calibration. *arXiv preprint arXiv:2601.15778*, 2026.

[34] William Zhang, Saurabh Amin, and Georgia Perakis. Decomposition-based modular conformal prediction for two-stage modeling. *arXiv preprint arXiv:2510.04406*, 2025.

A Action rule and cost matrix

Table 4: The action rule $F(c_2, c_3)$. *Unclear* is checked first.

$c_2 \setminus c_3$	0	1	2
1 (yes)	accept	defer	escalate
0 (no)	accept	accept	escalate
unclear	request evidence (any c_3)		

Table 5: Frozen cost matrix $\text{cost}(a, a^*)$. One unit of human cost is added per escalation. "req. ev." = request evidence.

action \ target	accept	escalate	req. ev.	defer
accept	0	10	6	8
escalate	1	0	1	1
req. ev.	1	1	0	1
defer	1	1	1	0

B Reproducibility

All calls used `jev-1.13.0`. Real-data calls: Mozilla 2,424 (including a label-wording arm) plus a 252-call smoke test; kernel 2,160 plus a 216-call smoke test. All returned HTTP 200 and parsed. Every dataset, raw response, execution manifest and analysis is hash-locked in the accompanying repository, as are the pre-registration documents and their revision history. The figure data in this paper are generated from committed result files by `make_figure_data.py`.